Machine Learning Project

Tomato Diseases Classification via Convolutional Network

Huynh Ngoc Hoang Khang - 1651678

1. INTRODUCTION

Agriculture plays a major role in society, providing food and jobs for a large portion of the populations, contributing to the global economy.However, diseases present a major challenge to farmers, not only harming the plant but also spreading to the crops surrounding it. Therefore, it is vital to monitor and respond quickly to plant diseases. Due to the increasing sizes of farms, it is not feasible to manually monitor every plants. Automated system must be employed to ensure effective responses.

This project aims to present a solution to detect disease on a plant, in this case the tomato, based on image of its leaf. Using neural networks and transfer learning, we can extract the features from an image and from there classify based on ten classes.

2. METHODOLOGY

2.1. Data Acquisition and Processing

Plant Village is a public dataset created by David Hughes and Marcel Salathé, containing 54303 leaf images and divided into 38 categories based on diseases and species in a variety of formats. For this project, the images from the tomato plants will be sufficient. The tomato dataset contains ten subsets, are in full color, with their original background, at a resolution of 256×256.

For the process of processing images and model training, we have chosen Pytorch library for its support of GPU integration on Window, allowing for faster training, and clear, well-maintained documentation. The images are first converted to the correct input size, which is 224×224, normalize, and converted into a tensor for model input. All of the images are loaded into a dataset and divided into a training and testing data using an 80/20 split.

2.2. Feature Extraction

Feature extraction is performed via convolutional layers. By convolving the image tensors with kernels, we can extract distinct features from the images and create various feature maps. These feature maps are of smaller sizes and therefore are less computationally demanding to process.

While initially a custom Convolutional Neural Network(CNN) was developed, a decision was made to adopt a transfer learning approach, utilizing the pretrained EfficientNetB0 model. EfficientNetB0 is trained upon a much larger and more diverse dataset, allowing for better feature extraction. Furthermore, because EfficientNetB0 is already pre-trained, we can freeze the base layers, allowing for faster training time and preventing overfitting.

2.3. Classification

The multi-channel features maps of size $C \times H \times W$ are flattened into 1d vector, which is then passed through a fully connected layer of 512 neurons. The final fully connected layer contains 10 neurons, corresponding to the 10 target classes.

2.4. GUI

The GUI is built using the Python CustomTKinter library, allowing the user to train, test the model using user uploaded images, and also see its performance metrics using the test dataset.

3. RESULT

Table 1 shows the performance of the model, including precision, recall, and F1-Score.

Table 1. Model performance metrics

Class	Precision	Recall	F1-score
Bacterial spot	98.60 %	98.60 %	98.60 %
Early blight	94.85 %	94.85 %	94.85 %
Late blight	98.52 %	98.52 %	98.52 %
Leaf Mold	98.53 %	98.53 %	98.53 %
Septoria leaf spot	98.22 %	98.22 %	98.22 %
Spider mites (Two-spotted)	97.84 %	97.84 %	97.84 %
Target Spot	95.77 %	95.77 %	95.77 %
Yellow Leaf Curl Virus	99.54 %	99.54 %	99.54 %
Mosaic virus	98.57 %	98.57 %	98.57 %
Healthy	99.05 %	99.05 %	99.05 %

Overall, the model performed relatively well, achieving an overall accuracy of 98.38%. This is a marked improvement in comparision to a custom-made CNN model, which averages around 93% accuracy.

As shown by the performance metrics table, there is a balance between precision and recall among all disease categories. The "Tomato Yellow Leaf Curl Virus" class achieved the highest F1-Score of 99.54%, followed closely by "Healthy" 99.05%. The lowest, yet still satisfactory, F1-Scores were for "Early Blight" and "Target Spot" at 94.85% and 95.77% respectively.

Fig. 1 shows the confusion matrix of the model.

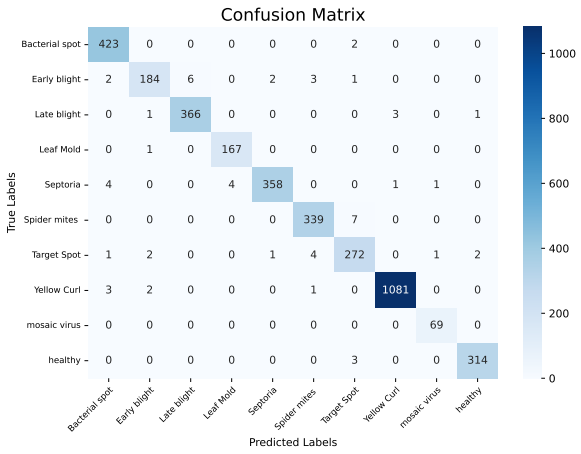

Figure 1. Confusion Matrix

However. when the model was tested informally on non-dataset image via the GUI, the result is noticeably less consistent. The model produced mixed results, with a drop inaccuracy and confidences when given images taken under real-world conditions. This could suggest that the model is tuned to ideal conditions such as clean background or when only a single leaf shown. Variations in real-world contribute to less reliable predictions.

4. CONCLUSION

This project has developed a plant health classification using Convolution Neural Network. By implementing a transfer learning approach, feature maps are extracted from images and fed into fully connected layers to classify them as one out of ten possible classes.

The CNN model yielded strong result, achieving over 98.38% overall accuracy on the Plant Village dataset. For classifying real images from non-ideal images, the model performs less accurately, possibly due to the variations and noises.

By enabling farmers to identify diseases early and accurately using a simple image, it can significantly reduce diagnosis time, leading to more timely interventions. This translates to improved crop yields and a reduction in costs for both farmers and consumers.

Further developments for this project may be explored:

- More diverse and robust data
- Expansion to classifying other plants
- Improvement on the GUI, implementation of threadings and training options

REFERENCES

[1] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing", *CoRR*, vol. abs/1511.08060, 2015. arXiv: 1511.08060. [Online]. Available: http://arxiv.org/abs/1511.08060

[2] M. Tan and Q. V. Le, *Efficientnet: Rethinking model scaling for convolutional neural networks*, 2020. arXiv: 1905.11946 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1905.11946

[3] S. Subramanian, S. Juarez, C. Breviu, D. Soshnikov, and A. Bornstein, *Learn the basics — pytorch tutorials*, https://docs.pytorch.org/tutorials/beginner/basics/intro.html, 2025.